
Exercice 1 : Encapsulation et `@property` (Niveau Facile)

Énoncé :

Crée une classe `CompteEpargne` qui gère un compte bancaire avec un taux d'intérêt.

1. La classe doit stocker le `titulaire` (public) et le `solde` (protégé `_solde`).
2. Utilise le décorateur `@property` pour rendre le `solde` accessible en lecture seule depuis l'extérieur.
3. Ajoute une méthode `deposer(montant: float)` qui valide que le montant est strictement positif avant de l'ajouter au solde.
4. Ajoute une méthode `appliquer_interet(taux: float)` (ex: `0.03` pour 3 %) qui calcule et ajoute les intérêts au solde.

Exercice 2 : Méthodes de classe et Factory (`@classmethod`) (Niveau Intermédiaire)

Énoncé :

Crée une classe `Employe` avec :

1. Les attributs d'instance `nom` (str) et `salaire_annuel` (float).
2. Une propriété `salaire_mensuel` calculée à la volée (`salaire_annuel / 12`).
3. Une méthode de classe (`@classmethod`) nommée `depuis_chaine(cls, chaine_csv: str)` qui permet de créer un employé à partir d'une chaîne au format "`Nom,SalaireAnnuel`" (ex: "`Alice,48000`").

Exercice 3 : Héritage et Méthodes Magiques (`Dunder Methods`) (Niveau Avancé)

Énoncé :

On souhaite gérer un panier de commande composé de différents produits.

1. Crée une classe `Produit` avec `nom` (str) et `prix` (float).
 - Implémente `__repr__` pour afficher `Produit('Nom', Prix)`.
2. Crée une classe `Panier` qui contient une liste interne de produits (`_produits`).
 - Implémente `__len__` pour retourner le nombre de produits dans le panier.
 - Implémente `__getitem__` pour permettre d'accéder à un produit par son index (`panier[0]`).
 - Implémente `__add__` pour fusionner deux objets `Panier` en un nouveau `Panier` contenant tous leurs produits.
 - Ajoute une méthode `calculer_total()` qui renvoie la somme des prix des produits.

Solutions et Corrections

Correction Exercice 1

```
class CompteEpargne:

    def __init__(self, titulaire: str, solde_initial: float = 0.0):
        self.titulaire = titulaire
        if solde_initial < 0:
            raise ValueError("Le solde initial ne peut pas être négatif.")
        self._solde = solde_initial

    @property
    def solde(self) -> float:
        """Accès en lecture seule au solde."""
        return self._solde

    def deposer(self, montant: float) -> None:
        if montant <= 0:
            raise ValueError("Le montant du dépôt doit être strictement positif.")
        self._solde += montant

    def appliquer_interet(self, taux: float) -> None:
        if taux <= 0:
            raise ValueError("Le taux d'intérêt doit être positif.")
        interets = self._solde * taux
        self._solde += interets

# Test
compte = CompteEpargne("Alice", 1000.0)
compte.deposer(500.0)
compte.appliquer_interet(0.03)  # +3%

print(f"Titulaire : {compte.titulaire}")
print(f"Solde final : {compte.solde:.2f} €")  # 1545.00 €

# Tentative de modification directe :
# compte.solde = 2000  # Lève une AttributeError
```

Correction Exercice 2

```
class Employe:

    def __init__(self, nom: str, salaire_annuel: float):
        self.nom = nom
        self.salaire_annuel = salaire_annuel

    @property
    def salaire_mensuel(self) -> float:
        return self.salaire_annuel / 12
```

```

    @classmethod
    def depuis_chaine(cls, chaine_csv: str) -> "Employe":
        """Factory method : instancie la classe à partir d'une ligne
        CSV."""
        nom, salaire_str = chaine_csv.split(",")
        return cls(nom.strip(), float(salaire_str.strip()))

# Test
emp1 = Employe("Bob", 60000.0)
emp2 = Employe.depuis_chaine("Charlie, 48000")

print(f"{emp1.nom} gagne {emp1.salaire_mensuel:.2f} € / mois")      # 5000.00
€ / mois
print(f"{emp2.nom} gagne {emp2.salaire_mensuel:.2f} € / mois")      # 4000.00
€ / mois

```

Correction Exercice 3

```

class Produit:

    def __init__(self, nom: str, prix: float):
        self.nom = nom
        self.prix = prix

    def __repr__(self) -> str:
        return f"Produit('{self.nom}', {self.prix})"

class Panier:

    def __init__(self, produits: list[Produit] | None = None):
        self._produits: list[Produit] = produits if produits is not None
    else []

    def ajouter(self, produit: Produit) -> None:
        self._produits.append(produit)

    def calculer_total(self) -> float:
        return sum(p.prix for p in self._produits)

# Dunder Methods
    def __len__(self) -> int:
        return len(self._produits)

    def __getitem__(self, index: int) -> Produit:
        return self._produits[index]

```

```
def __add__(self, autre_panier: "Panier") -> "Panier":
    if not isinstance(autre_panier, Panier):
        return NotImplemented
    # Fusion des deux listes de produits dans un nouveau Panier
    return Panier(self._produits + autre_panier._produits)

# Test
p1 = Produit("Écran", 200.0)
p2 = Produit("Souris", 30.0)
p3 = Produit("Clavier", 70.0)

panier_a = Panier([p1, p2])
panier_b = Panier([p3])

# Utilisation de __len__ et __getitem__
print(f"Nombre d'articles dans Panier A : {len(panier_a)}") # 2
print(f"Premier article du Panier A : {panier_a[0]}")      #
Produit('Écran', 200.0)

# Utilisation de __add__ pour fusionner les paniers
panier_combine = panier_a + panier_b
print(f"Articles du panier combiné : {len(panier_combine)}") # 3
print(f"Total combiné : {panier_combine.calculer_total():.2f} €") # 300.00
€
```